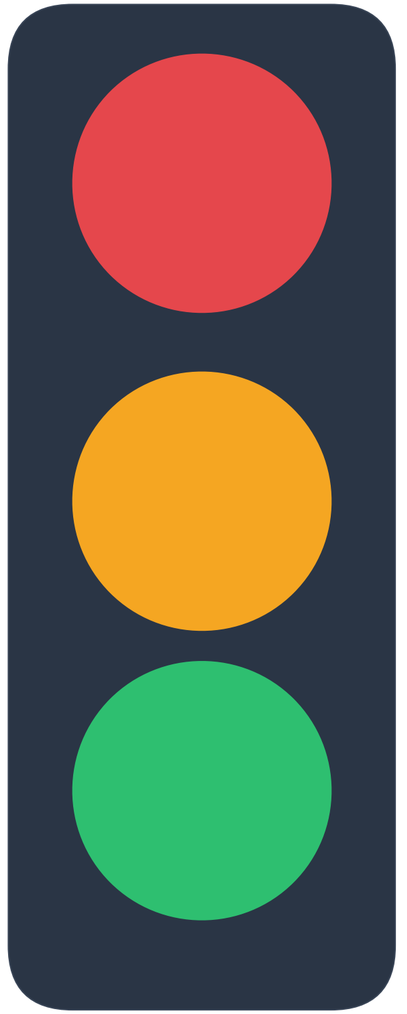
Real-Time Operating System for a Traffic Signal Controller

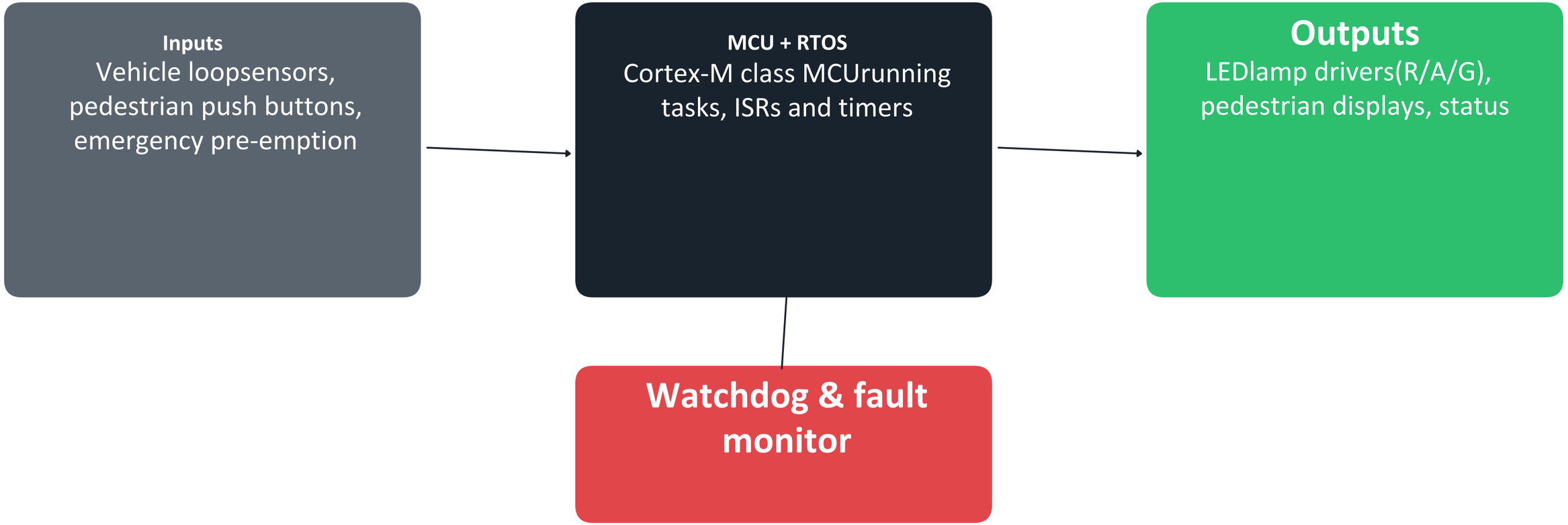
OS type & justification • OS structure & components • System

calls for process control and timing

name:Christina .y
reg no:1965260124
course code :CSA0408



The System: Embedded Traffic Signal Control



Why it matters: a missed deadline is not just slow — it can show conflicting greens. Timing must be guaranteed, not best-effort.

OS Type: Hard Real-Time OS (RTOS)

Chosen OS: FreeRTOS on an ARM Cortex-M MCU (VxWorks / QNX

real-time

Hard

for certified systems)

1

Deterministic timing

Preemptivepriorityscheduler gives bounded, repeatable response.

2

Low interrupt latency

Pedestrian / emergency inputs handled within microseconds.

3

Small footprint

Kernel needs only a few KB of ROM and RAM on a low-cost MCU.

4

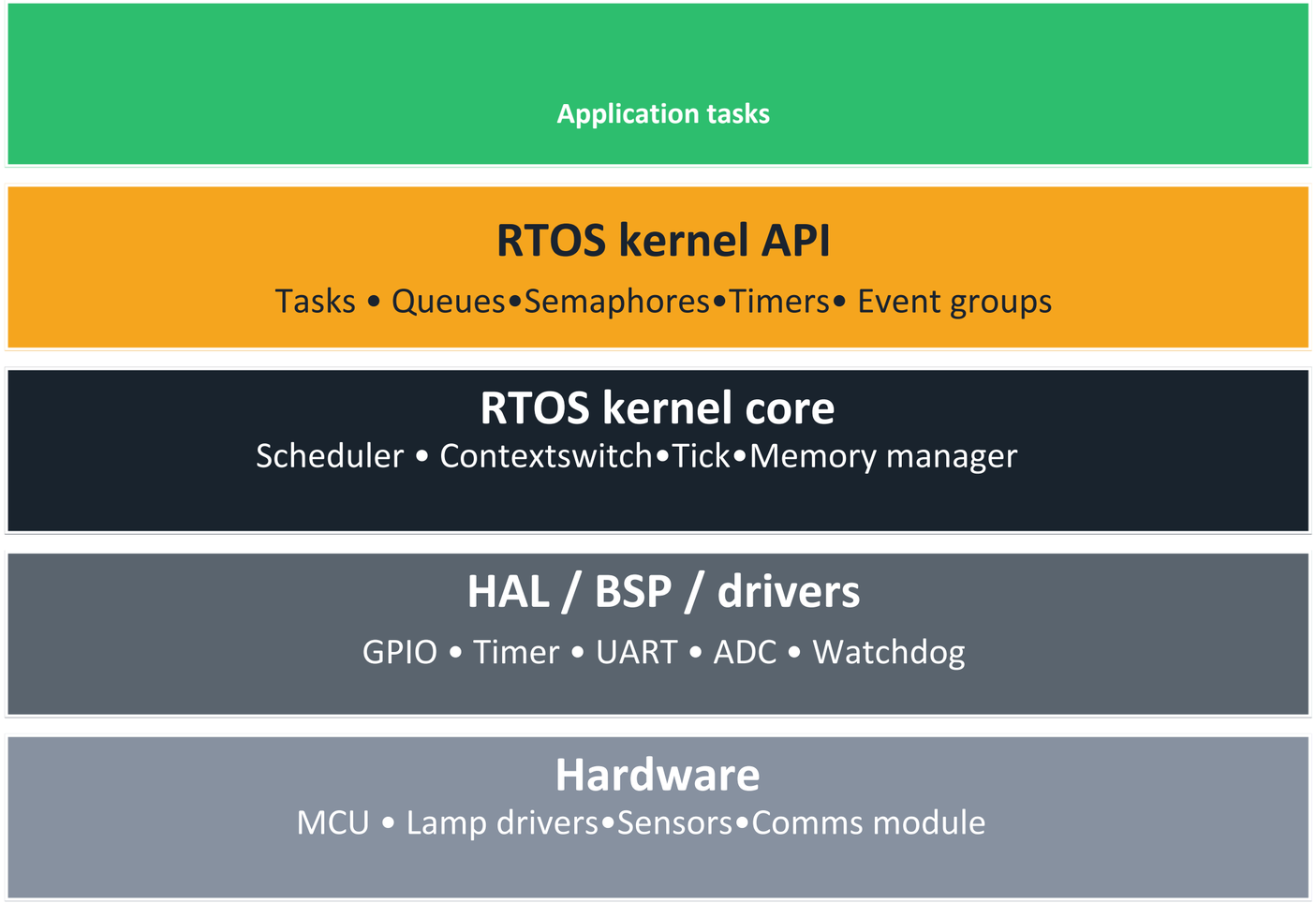
Safety & reliability

No unbounded delays, no page faults; easy to analyse and certify.

Justification: Why Not a General-Purpose OS?

Criterion	GeneralOS (Linux/Windows)	RTOS (FreeRTOS / QNX)
Scheduling	Fair-share; timingnotguaranteed	Priority preemptive; deadlines guaranteed
Interrupt latency	Variable, milliseconds	Bounded, microseconds
Memory	Virtual memory, paging (non-deterministic)	Static / fixed-block allocation
Footprint	Hundreds of MB	Few KB to a few MB
Boot & recovery	Slow boot, heavy	Boots in ms; fast watchdog reset
Fit for traffic control	Poor — risk of missed deadlines	Excellent — predictable and safe

OS Structure: Layered Architecture



Microkernel-style, layered design

Onlytheessentials run inthe kernel; everything else is a task above it.

Benefits

- Portable: only HAL changes per MCU
- Modular: tasks are independent
- Small, analysable kernel

Key OS Components

Scheduler

Preemptive, fixed-priority; round-robin among equals.

Task manager

Creates tasks with TCB, stack and priority; tracks states.

IPC & sync

Queues, binary / counting semaphores, mutexes, event groups.

Timer services

Tick timer plus software timers for phase durations.

Interrupt handling

Short ISRs defer work to tasks via FromISR calls.

Memory manager

Static allocation or fixed heap; no paging.

Task Design and Priorities

Task	Priority	Period / trigger	Role
Safety monitor	Highest	10ms	Detects conflicting greens, forces flashing red
Pedestrian / emergency ISR task	High	Event (interrupt)	Handles button press and pre-emption requests
Signal FSM	Medium-high	Event + timer	Controls R / A / G phase transitions
Sensor polling	Medium	50 ms	Reads vehicle loop detectors Reports to central control, logs faults
Comms / logging	Low	1 s	

Rate-monotonic style: the shorter the deadline, the higher the priority.

System Calls: Process (Task) Control

	POSIX analogue	Useintraffic controller
`xTaskCreate()`	fork() / pthread_create()	Createsignal, sensor, safety tasks at start-up
`vTaskDelete()`	exit() / pthread_exit()	Remove a task no longer needed
`vTaskSuspend() / Resume()`	kill(SIGSTOP / SIGCONT)	Pause normal cycle during flashing mode
`vTaskPrioritySet()`	setpriority()	Raise priority for emergency vehicle
`xQueueSend() / Receive()`	msgsnd() / msgrcv()	Pass sensor events to the signal FSM
`xSemaphoreTake() / Give()`	sem_wait() / sem_post()	Protect shared lamp-state data

System Calls: Timing Control

	POSIX analogue	Use in traffic controller
`vTaskDelay()`	sleep() / nanosleep()	Relative delay, e.g. short amber hold
`vTaskDelayUntil()`	clock_nanosleep(ABSTIME)	Drift-free periodic loop for the signal FSM
`xTaskGetTickCount()`	clock_gettime()	Time-stamp events and measure phase length
`xTimerCreate() / Start()`	timer_create() / timer_settime()	Software timer for green / red durations
`vTaskSetTimeOutState()`	select() timeout	Timeout on sensor or comms reply
`xTaskNotifyFromISR()`	signal handler	Wake task from tick or button interrupt

Putting It Together: Signal Control Task

```
void vSignalTask(void*p){
 TickType_t last=xTaskGetTickCount();
for (;;) {
   setLamp(GREEN);
   vTaskDelayUntil(&last, pdMS_TO_TICKS(30000));
   setLamp(AMBER);
   vTaskDelayUntil(&last, pdMS_TO_TICKS(3000));
   setLamp(RED);
   vTaskDelayUntil(&last, pdMS_TO_TICKS(33000));
 }
}
// main: xTaskCreate(vSignalTask, "SIG", 256,
//         NULL, 3, NULL); vTaskStartScheduler();
```

Cycle timeline

Green 30 s

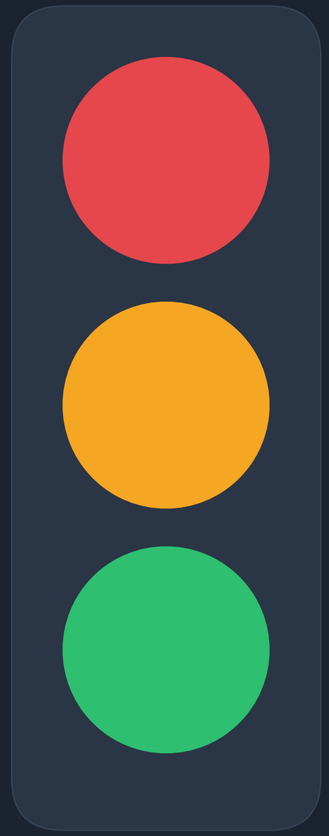
Amber 3 s

Red 33 s

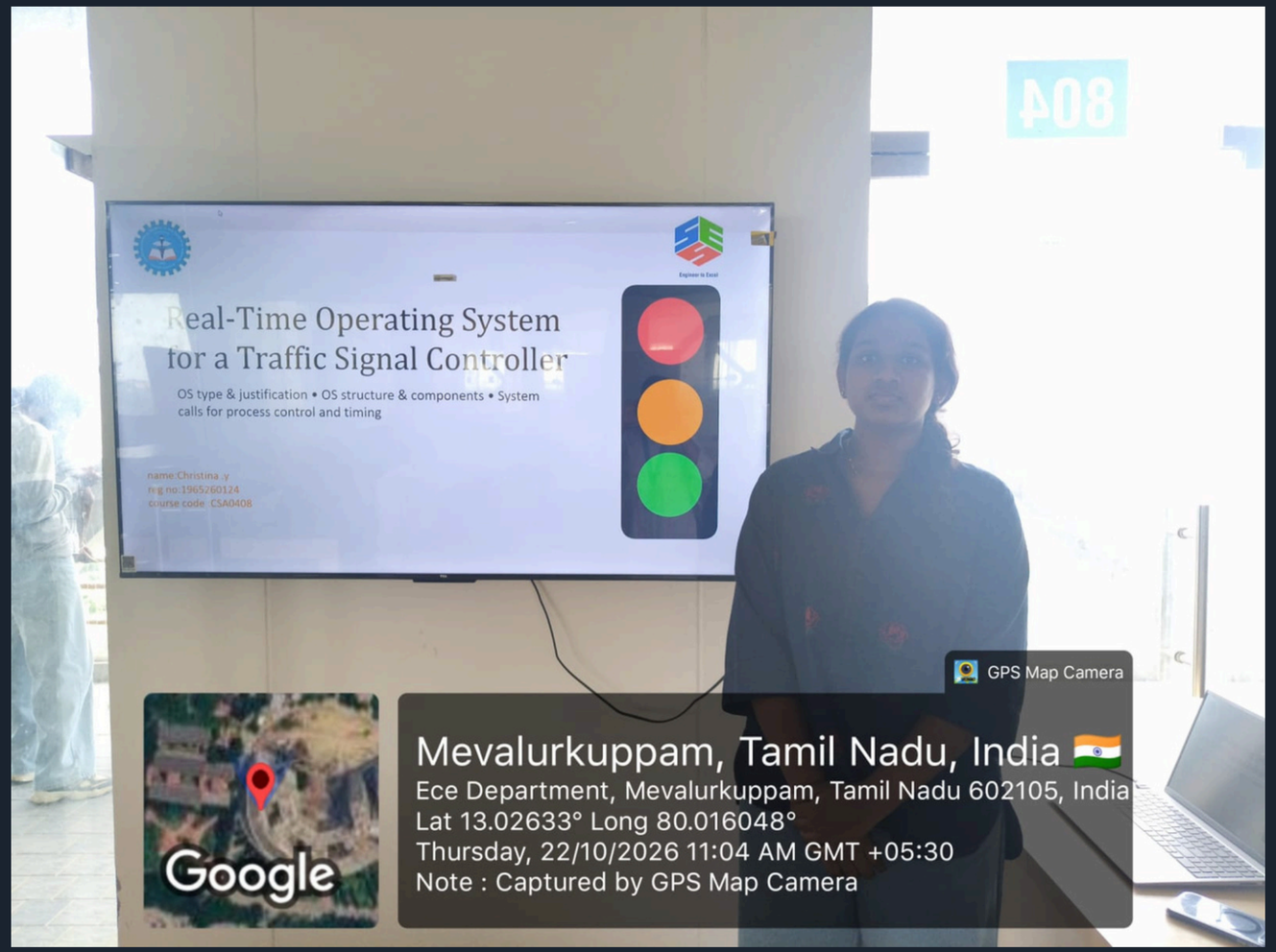
vTaskDelayUntil() keeps the period exact, with no drift from execution time.

Summary

- HardRTOS(FreeRTOS) gives guaranteed, deterministic timingforsafety-critical signals.
- Layered structure: application tasks, kernel API and core, HAL, hardware.
- Task-control and timing system calls build a drift-free, fault-safe signal cycle.



Thank you — Questions?


Real-Time Operating System
for a Traffic Signal Controller
OS type & justification • OS structure & components • System calls for process control and timing
name:Christina .y
reg no:1965260124
course code :CSA0408
GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
Ece Department, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.02633° Long 80.016048°
Thursday, 22/10/2026 11:04 AM GMT +05:30
Note : Captured by GPS Map Camera
Google